Questions for a Review Process

Questions to ask and what to do with the answers when starting a peer review process.

"Every development process is different." Maybe, but successful peer review deployments do have common elements. No matter how you tailor your process, you should be aware of the typical issues and pitfalls so you're making informed decisions.

What are the goals of review?

The answer might seem obvious, but it is perhaps the most important question to answer, and it must be answered in a certain way.

There are many possible benefits of code review. You'll find more defects in development. Your code will be well-documented. You'll be able to mentor and direct new hires while allowing them to jump into the code quickly.

No matter what your primary goals are, some of these effects will happen. But if you're not able to define and measure a specific goal

you won't be able to judge whether peer review is really achieving the results you require.

For one of our customers, the goal was to reduce support costs. In particular, it cost them $33 per customer support call, and the company wanted to reduce the number of calls per year from 50,000. Code review was used both to remove defects (incorrect behavior or errant error dialogs would generate calls) and to improve usability (if you can't figure out how to do something, you call support). Over a few years the support calls were down to 20,000 even with a 2-fold increase in product sales, representing $2.6 million in savings[1].

For another customer, the goal was to reduce the number of defects introduced during development. The metrics group had a phase-injection chart[2], and the numbers for development didn't look good. Most of the defects found by QA were caused by faulty code, not a result of bad design or incorrect requirements. And development's numbers were even worse for defects found by end users. The goal of code review was simple: To reduce the number of defects injected by development. Fortunately, with a good metrics process already in place and using a tool-assisted code review process that tracked defect metrics automatically, it was easy for them to directly measure their progress and to attempt various review techniques until they found a process that uncovered the majority of the defects while minimizing time spent performing the reviews. Code review eliminated defects released from development to QA, just as design and architecture meetings eliminate defects released from design to development.

The thing to notice is that in both cases the answer to the question "What is the goal of the review" was a directly measurable

---

[1] With doubled sales, the expected number of support calls is 100,000. The savings is the cost of the 80,000 calls that never occurred.

[2] A phase-injection chart is a tool that shows (a) in which development phase each defect was "injected" (introduced) and (b) in which development phase each defect was found. The chart can answer questions like "How effective is QA at finding defects before customers do?" and "How often are incorrect requirements at fault rather than developer error" and "How many errors are caused by faulty code rather than bad design?" See the first essay in this collection for an example.

quantity, not an ambiguous statement like "fix more bugs." You must have a quantifiable measure of success or else you cannot know if your review process is making a difference. "Number of support calls per month" or "Percent of defects injected by development" are measurable values.

In many cases having a measurable goal also means you can assign a dollar value to achieving the goal. This can be useful when convincing the powers-at-be that installing a review process makes business sense. Even without a monetary value, just having a measurable goal makes it easier to pitch the idea because you're holding yourself accountable. If it doesn't work out, everyone will see that, and you'll try something else.

It also helps when you're ready to tweak the process. You'll want to change the checklists, change how often you perform reviews, change which code you require to have reviews, change how much time you spend reviewing, and so on. Which changes make things better? Or worse? Only if the results are measurable can you answer these questions with confidence.

The goal doesn't have to be complicated. For example, if your general goal is to fix more defects, make an informal scan of the last several hundred bugs and determine how many of them you believe should have been prevented had you had a code review process in place. Then you can make a goal such as "Reduce number of reported defects by 50%" or "Fix at least as many defects in code review as are reported by QA and customers together."

How amenable are your developers to review?

We see two distinct reactions when management announces a new code review policy. In some groups they realize their defects are out of control, they're scared to make code changes, or they know some new group is making worrisome changes willy-nilly in their code base. These groups want review because they know it will address their existing complaints.

The other (more common) reaction is negative. "Oh great, more process." "Now I'll have to do more work just to check something in to ClearCase." "I wonder if we'll be fired if our code has too many defects." This is typical with developers who have never done code review or who have had irresponsible managers in the past.

In this case you have several options. The "baby-steps" approach starts with something easy to swallow. For example, give everyone the tools for code review, but don't require reviews. Just suggest that whenever a developer believes a review is warranted, get someone else to review it. Use your judgment and don't worry about metrics. Just use this as another tool to help keep the code base solid. New hires should use this much more often to double-check that they didn't break anything unknowingly.

Once the group gets used to the idea of review, they'll also start seeing some of the positive social and personal aspects of review[3], and they'll see that it does contribute positively to the development process. Then you can start enforcing review in certain cases – all changes on a stable branch, or in a certain code module, or when the change is particularly complex. It's easy to get developers on board with that idea once they agree that code review is helpful in the first place.

Another baby-step is to have "random drug test" reviews. Just take version control check-ins at random and review them. One advantage of this is that you can control exactly how much time is spent performing reviews. Another is that you'll get higher code quality across the board because of the Ego Effect[4].

Eventually you can roll out review on a wide basis. Depending on the goals of review, you might not need to review every single code

---

[3] When done correctly, code review can be personally rewarding and fun. See "Social Aspects of Review" elsewhere in this essay collection.

[4] The "Ego Effect" is that just knowing that someone else might look at your code is enough to get developers to self-check their work and generally be more careful. It is fully described in the "Social Aspects of Review" essay elsewhere in this collection.

change; use your measurement of success to impose just enough review to achieve the goal.

How will you improve the process over time?

There are many reasons why the peer review process is dynamic. No process is perfect when it first rolls out. Software development changes over the lifetime of the project. Development techniques change over time.

It's not a question of whether the process will change; it's a question of how you will know what needs changing and how you will measure the effect of your changes.

To measure any process you need metrics. In the case of code review there are two kinds: external metrics that measure the overall effectiveness of the process and internal metrics that measure rates and values coming from the act of review.

The external metrics are those wrapped up in your definition of the goal of the review. In our earlier examples these were "number of monthly support requests" or "percentage of defects injected by development." These are the ultimate measure of the success of the code review program, and these are the numbers to watch over time. If your initial results are good but you start slipping, this is indicates the process needs changing; after a change, an improvement in these metrics are the final proof that the change was successful.

Although ultimately the external metrics are all that matter, frequently the feedback is slow and blind. For example, to determine whether a new process affected the number of support calls, you have to wait until the new version is released, then wait for customers to get their hands on it, then wait for enough calls to start coming in to see if there was any effect. Meanwhile many months have passed, and in the interim you have zero feedback.

This is where the internal metrics come in. The most common of these are:

- Inspection Rate. "How fast are we able to review code?" Normally measured in kLOC/man-hour.
- Defect Rate. "How fast are we able to find defects?" Normally measured in defects/man-hour.
- Defect Density. "How many defects do we expect to find in a given amount of code?" Normally measured in defects/kLOC.

The exact values of these measures depend on many factors including the type of development (new code, maintenance), stability of the code (final, beta, development), complexity of the changes, experience of the developers and reviewers, programming tools and languages, and amount of time spent in review. As with any metric, the important thing is to compare apples with apples – as you watch the process change over time, compare changes of similar complexity in similar code modules within the same development groups.

As you change the process, watch these numbers. Decide ahead of time what you'd expect to happen. For example, say you decide you need to do a better job keeping defects out of the stable code branch. So you change the process with the intention of having more reviewers look at any stable branch changes. You expect the inspection rate to drop (because more people are spending more time on every change) and you expect the defect density to increase (because for a given change you're wanting more defects to be uncovered). These are values you can measure quickly – over the space of even just ten reviews or a few weeks.

Although the external metrics are the only true indicator of the success of a change to the process, internal metrics give you fast feedback on how well you're achieving the more immediate goals.

There are several other essays in this collection that describe the types of metrics you can collect, what those metrics mean, what they don't mean, what other development groups have done with them, and what the published literature tells us.

How will you collect metrics?

Developers hate collecting metrics. And for good reason: no one wants to do a code review holding a stopwatch and wielding line-counting tools.

Even if you try to mandate such things, people forget to do it. In studies with our customers as well as in observing our own development process, we found that "stop-watch" style measurement is extremely inaccurate. During a review there might be in interruption – a phone call, an e-mail, or even a bathroom break. Frequently you forget to pause the timer. When you get back, sometimes you forget to account for the missing time, and even when you do it's often a guess.

And it gets worse! Our field experience is that developers often just make up numbers that they believe make them look good. The irony is that often the developer doesn't change the metric to his favor because he incorrectly assumes what a "good" value is. For example, the most common cheat is to report less time-in-review than was actually spent. The developer believes this is good because it shows he is more efficient at finding defects. But studies have repeatedly shown that only by spending *more* time in review will you find more defects! Another common cheat goes the other way – to report more time spent on review than actually took place to appear as though the review has happened when in fact the developer is shirking his responsibility. Again, the mismatch between defect density and time-on-task exposes the cheat.

Only an automated or tightly controlled process can give you repeatable metrics. Clearly line-counting tools should be used for lines-changed and lines-inspected measurements.

Automated timing systems have to be intelligent to be effective. Just putting "start" and "stop" buttons in a computer interface does not solve the stopwatch problem. It might get your numbers into a database, but you still have the problems of dealing with interruptions or with off-line activity (e.g. whiteboard discussions).

Your timing system needs to be smart enough to account for these external factors. Being inaccurate even by 15 minutes can throw off your rate metrics by an order of magnitude[5].

How will you manage negative emotions?

Criticism is personal. No matter how thick-skinned you are, when you put out your best effort and someone else finds fault, it's not a nice feeling. Some people can put away those feelings and accept the process as being good for the project as well as an opportunity for personal improvement, but inevitably there will be strife.

Another different problem arises with any process that involves collecting metrics or audit trails. Developers, through misconceptions or experiences with unreasonable managers, often feel like "Big Brother" is watching. The biggest concern is whether code review metrics will impact their performance evaluations.

At the very least there are specific things managers can do in development meetings to mitigate these feelings. We've devoted a whole essay to the subject.

Some groups take this one step further and have professional training sessions with their developers and reviewers. Everyone can learn how to accept criticism without taking it personally, and reviewers can learn techniques and language that lessens these feelings. Unfortunately, our experience is that even professional training cannot overcome the illogic of human emotion, so the techniques given in the "Social Aspects" chapter should still be employed.

---

[5] Blatant Plug: At SmartBear we performed empirical tests (with stopwatches at customer sites) to see if activity logs in our web-based code review tool could be used to predict "actual time on task" even including external factors. We developed a heuristic that correlated very well with reality. The result is we can compute "man-hours on task" without any human input (no start/stop buttons) accurately enough for metrics analysis. This is a part of our CodeCollaborator product described elsewhere in this essay collection.

How will you develop the review checklist?

The checklist is an important component of any review. There are several ways to build a checklist; you'll want to determine which technique is most appropriate for your development group.

Checklists are most effective at detecting omissions. Omissions are typically the most difficult types of errors to find. After all, you don't need a checklist to tell a reviewer to look for algorithm errors or sensible documentation – that's what a reviewer will naturally do when confronted with source code. The difficult thing is to notice when something is *not* there. For example, the checklist can remind the reviewer to confirm that all errors are handled, that function arguments are tested for invalid values, and that unit tests have been created. If the author forgot it, the reviewer is likely to forget it as well.

In fact, as long as the reviewer has a checklist, shouldn't the author have it as well? Why shouldn't the author get the same reminders? Of course authors should, which is why checklists should be stored in easy-to-access places such as shared drives or an intranet or even checked into version control along with the source code. This doesn't relieve the reviewer of his duties but it does help the author get his changes through review on the first pass.

In a more extreme case of this, the PSP[6] suggests that each author should have his own personal checklist. The SEI points out that each person make 15-20 common mistakes; if you notice what they are (through peer review) you can develop your own personal checklist. Authors can even give their personal checklist to reviewers.

The content of the checklist will depend on the purpose of the review. A security audit will have items such as "All input data is cleansed before use" and "All invalid input data is rejected." A style review will contain items such as "All methods are documented

---

[6] The PSP (Personal Software Process) and SEI (Software Engineering Institute) with their policies on code review and checklists is fully described in its own essay in this collection.

according to company standards" or "All private member fields start with an underscore."

We are frequently asked for examples of items that make sense on a checklist. The following is a guide that should get you thinking about the various categories of items that might appear on the checklist. Keep in mind that the longer the checklist gets the less effective each item is, so try to keep it down to the 10-20 items that you find most often wrong.

Sample Checklist Items

1. Documentation: All subroutines are commented in clear language.
2. Documentation: Describe what happens with corner-case input.
3. Documentation: Complex algorithms are explained and justified.
4. Documentation: Code that depends on non-obvious behavior in external libraries is documented with reference to external documentation.
5. Documentation: Units of measurement are documented for numeric values.
6. Documentation: Incomplete code is indicated with appropriate distinctive markers (e.g. "TODO" or "FIXME").
7. Documentation: User-facing documentation is updated (online help, contextual help, tool-tips, version history).
8. Testing: Unit tests are added for new code paths or behaviors.
9. Testing: Unit tests cover errors and invalid parameter cases.
10. Testing: Unit tests demonstrate the algorithm is performing as documented.
11. Testing: Possible null pointers always checked before use.
12. Testing: Array indexes checked to avoid out-of-bound errors.
13. Testing: Don't write new code that is already implemented in an existing, tested API.
14. Testing: New code fixes/implements the issue in question.

15. Error Handling: Invalid parameter values are handled properly early in the subroutine.
16. Error Handling: Error values of null pointers from subroutine invocations are checked.
17. Error Handling: Error handlers clean up state and resources no matter where an error occurs.
18. Error Handling: Memory is released, resources are closed, and reference counters are managed under both error and non-error conditions.
19. Thread Safety: Global variables are protected by locks or locking subroutines.
20. Thread Safety: Objects accessed by multiple threads are accessed only through a lock.
21. Thread Safety: Locks must be acquired and released in the right order to prevent deadlocks, even in error-handling code.
22. Performance: Objects are duplicated only when necessary.
23. Performance: No busy-wait loops instead of proper thread-synchronization methods.
24. Performance: Memory usage is acceptable even with large inputs.
25. Performance: Optimization that makes code harder to read should only be implemented if a profiler or other tool has indicated that the routine stands to gain from optimization. These kinds of optimizations should be well-documented and code that performs the same task simply should be preserved somewhere.

The most effective way to build and maintain your checklist is to match defects found during review to the associated checklist item (if any). Items that turn up many defects should be kept. Defects that aren't associated with any checklist item should be scanned periodically. Usually there are categorical trends in your defects; turn each type of defect into a checklist item that would cause the reviewer to find it.

Over time your team will become used to the more common checklist items and will adopt programming habits that prevent some of them altogether. You can accelerate and codify this by reviewing the "Top 5 Most Violated" checklist items every month to determine whether anything can be done to help developers avoid the problem. For example, if a common problem is "not all methods are fully documented," you might be able to enable a feature in your IDE that requires developers to have at least some sort of documentation on every method.

Stay vigilant

The common theme to all these suggestions is to keep an eye on things. Of course your first attempt at code review won't be perfect, but with a few measurements and occasional process review you can tune your methods over time. And even the best process shouldn't stagnate as the needs of the development group change.

Make a conscientious effort to pay attention to how things are going, keep the dialog open with everyone involved, and keep your eye on your clearly-stated goal for success.